FE515 Lecture 9: Black-Scholes and Implied Volatility

Sean O'Leary

Contents

1 Learning goals **1**

2 Black-Scholes model **2**

2.1 Calculate a call price 2

2.2 Turn the formula into a function 3

3 Implied volatility: reversing Black-Scholes **3**

3.1 Bisection 3

4 Vega and Newton-Raphson **5**

4.1 Newton-Raphson 5

4.2 A function we can use with market prices 6

5 Implied volatility from SPY options **6**

5.1 Get the stock price and interest rate 6

5.2 Choose an expiration and inspect the option chain 7

5.3 Implied volatility for one option 8

5.4 Implied volatility across strikes 8

5.5 Plot the volatility smile 9

6 Summary **10**

1 Learning goals

By the end of class, students should be able to:

1. Price a European call using Black-Scholes.
2. Explain why implied volatility is a root-finding problem.
3. Use bisection and Newton-Raphson to estimate implied volatility.
4. Apply the same calculation to real option quotes and plot a volatility smile.

2 Black-Scholes model

The Black-Scholes model assumes that a stock follows geometric Brownian motion. Under the risk-neutral measure,

$$dS_t = rS_t\,dt + \sigma S_t\,dW_t^Q.$$

For a European call on a stock that pays no dividends, the payoff at expiration is

$$(S_T - K)^+ = \max(S_T - K, 0).$$

The Black-Scholes price is

$$C(S_0, K, T, \sigma, r) = S_0 N(d_1) - Ke^{-rT} N(d_2),$$

where

$$d_1 = \frac{\log(S_0/K) + (r + \sigma^2/2)T}{\sigma\sqrt{T}}, \qquad d_2 = d_1 - \sigma\sqrt{T}.$$

Here, S_0 is the current stock price, K is the strike, T is the time to expiration in years, σ is the annualized volatility, and r is the annual risk-free rate. Both σ and r are entered as decimals.

2.1 Calculate a call price

```
S0 <- 100
K <- 100
T1 <- 1
sigma <- 0.20
r <- 0.05

d1 <- (log(S0 / K) + (r + 0.5 * sigma^2) * T1) /
  (sigma * sqrt(T1))
d2 <- d1 - sigma * sqrt(T1)

call.price <- S0 * pnorm(d1) - K * exp(-r * T1) * pnorm(d2)
call.price
```

```
## [1] 10.45058
```

`pnorm()` gives the standard normal cumulative distribution function $N(\cdot)$. We use `T1` rather than `T` since `T` is also shorthand for `TRUE` in R.

2.2 Turn the formula into a function

We will need to calculate option prices repeatedly, so let's write a function instead of copying the formula each time.

```
bs.call <- function(S0, K, T1, sigma, r) {
  d1 <- (log(S0 / K) + (r + 0.5 * sigma^2) * T1) /
    (sigma * sqrt(T1))
  d2 <- d1 - sigma * sqrt(T1)

  S0 * pnorm(d1) - K * exp(-r * T1) * pnorm(d2)
}

bs.call(100, 100, 1, 0.20, 0.05)
```

```
## [1] 10.45058
```

3 Implied volatility: reversing Black-Scholes

Black-Scholes takes a volatility and gives us an option price. In the market, we already know the option price. **What volatility would make the model match that price?**

If the observed price is P, we want to solve

$$C(S_0, K, T, \sigma, r) = P.$$

Equivalently, define

$$f(\sigma) = C(S_0, K, T, \sigma, r) - P.$$

Implied volatility is the value of σ that makes $f(\sigma) = 0$. This is exactly the type of problem we studied in Lecture 8.

3.1 Bisection

Bisection starts with an interval $[a, b]$ where the function changes sign. It repeatedly cuts that interval in half. When there is no sign change, our function returns `NA` rather than stopping the entire calculation.

```
bisection <- function(f, a, b, tol = 1e-4, N.max = 100) {
  fa <- f(a)
  fb <- f(b)

  if (!is.finite(fa) || !is.finite(fb) || fa * fb > 0) {
    return(NA_real_)
  }

  for (n in 1:N.max) {
```

```
    mid <- (a + b) / 2
    fm <- f(mid)

    if (abs(b - a) < tol || abs(fm) < 1e-10) {
      return(mid)
    }

    if (fa * fm <= 0) {
      b <- mid
    } else {
      a <- mid
      fa <- fm
    }
  }

  NA_real_
}
```

Suppose we observe a call price of 10 with the same stock, strike, maturity, and interest rate from our example.

```
market.price <- 10

price.diff <- function(sigma) {
  bs.call(S0, K, T1, sigma, r) - market.price
}

price.diff(0.01)
```

```
## [1] -5.122942
```

```
price.diff(1)
```

```
## [1] 29.84016
```

```
sigma.bisection <- bisection(price.diff, a = 0.01, b = 2)
sigma.bisection
```

```
## [1] 0.1879692
```

```
bs.call(S0, K, T1, sigma.bisection, r)
```

```
## [1] 9.999909
```

The estimated volatility should give a Black-Scholes price close to our observed $10.

4 Vega and Newton-Raphson

Bisection only needs function values. Newton-Raphson also uses the derivative to decide how far to move. For implied volatility, the derivative of the option price with respect to volatility is called **Vega**:

$$\text{Vega} = \frac{\partial C}{\partial \sigma} = S_0 \phi(d_1)\sqrt{T}.$$

Here $\phi(\cdot)$ is the standard normal density, which R calculates with `dnorm()`.

```
Vega <- function(S0, K, T1, sigma, r) {
  d1 <- (log(S0 / K) + (r + 0.5 * sigma^2) * T1) /
    (sigma * sqrt(T1))

  S0 * dnorm(d1) * sqrt(T1)
}

Vega(S0, K, T1, sigma, r)
```

```
## [1] 37.52403
```

4.1 Newton-Raphson

From Lecture 8, the update is

$$\sigma_{n+1} = \sigma_n - \frac{f(\sigma_n)}{f'(\sigma_n)}.$$

In this problem $f'(\sigma)$ is Vega. We also stop if Vega is nearly zero or an update would produce a nonpositive volatility.

```
Newton_Raphson <- function(f, df, x0, tol = 1e-8, N.max = 100) {
  x <- x0

  for (n in 1:N.max) {
    fx <- f(x)
    dfx <- df(x)

    if (!is.finite(fx) || !is.finite(dfx) || abs(dfx) < 1e-10) {
      return(NA_real_)
    }

    x.new <- x - fx / dfx
    if (!is.finite(x.new) || x.new <= 0) return(NA_real_)
    if (abs(x.new - x) < tol) return(x.new)

    x <- x.new
  }
```

```
  NA_real_
}
```

Let's compare the two methods on the same option price.

```
sigma.newton <- Newton_Raphson(
  f = price.diff,
  df = function(sigma) Vega(S0, K, T1, sigma, r),
  x0 = 0.25
)

c(Bisection = sigma.bisection, Newton = sigma.newton)
```

```
## Bisection    Newton
## 0.1879692 0.1879716
```

They should be close. Newton-Raphson generally converges faster, but bisection is more reliable when we have a valid bracket.

4.2 A function we can use with market prices

For the rest of the lecture, we will use bisection. We only need to provide the contract information and the market price. The search interval below assumes volatility is between 0.01% and 300%; quotes without a root in that interval will return `NA`.

```
implied.vol.call <- function(S0, K, T1, r, price) {
  f <- function(sigma) bs.call(S0, K, T1, sigma, r) - price
  bisection(f, a = 0.0001, b = 3)
}
```

5 Implied volatility from SPY options

Now let's use actual option prices rather than our made-up $10 example. We estimate the traded option price using the bid-ask midpoint:

$$P = \frac{\text{Bid} + \text{Ask}}{2}.$$

One modeling simplification: SPY pays dividends and its options allow early exercise. Our Black-Scholes formula assumes neither. The implied volatilities below are therefore approximations, especially for deep in-the-money options.

5.1 Get the stock price and interest rate

```r
# install.packages("quantmod")  # Run once, if needed
library(quantmod)

SPY.S0 <- as.numeric(getQuote("SPY")$Last)

# Latest available three-month Treasury yield from FRED
DGS3MO <- getSymbols("DGS3MO", src = "FRED", auto.assign = FALSE)
r <- as.numeric(tail(na.omit(DGS3MO), 1)) / 100

c(Spot = SPY.S0, Risk_free_rate = r)
```

```
##           Spot Risk_free_rate
##       769.4800         0.0422
```

The Treasury yield is reported as a percentage, so we divide it by 100. We use it as an approximation of the continuously compounded risk-free rate in Black-Scholes.

5.2 Choose an expiration and inspect the option chain

We will use one expiration with at least 30 calendar days remaining. Very short-dated options can produce unstable implied-volatility estimates.

```r
SPY.options <- getOptionChain("SPY", Exp = NULL)

expiration.dates <- as.Date(names(SPY.options), format = "%b.%d.%Y")
days.remaining <- as.numeric(expiration.dates - Sys.Date())

eligible <- which(days.remaining >= 30)
if (length(eligible) == 0) stop("No expiration at least 30 days away.")

i <- eligible[which.min(days.remaining[eligible])]
selected.expiration <- expiration.dates[i]
T1 <- days.remaining[i] / 365

SPY.option <- SPY.options[[i]]$calls

c(Expiration = as.character(selected.expiration),
  Days_remaining = days.remaining[i])
```

```
##     Expiration Days_remaining
##   "2026-11-30"           "32"
```

To focus on reasonably relevant strikes, we'll keep calls within 20% of the current stock price and remove quotes with missing prices or invalid bid-ask spreads.

```r
SPY.option <- subset(
  SPY.option,
  is.finite(Strike) & is.finite(Bid) & is.finite(Ask) &
    Bid > 0 & Ask >= Bid &
    Strike >= 0.8 * SPY.S0 & Strike <= 1.2 * SPY.S0
)

SPY.option <- SPY.option[order(SPY.option$Strike), ]
SPY.option$Price <- (SPY.option$Bid + SPY.option$Ask) / 2

head(SPY.option[c("Strike", "Bid", "Ask", "Price")])
```

```
##    Strike    Bid    Ask   Price
## 30    617 145.82 148.53 147.175
## 31    620 150.65 153.32 151.985
## 34    630 140.77 143.40 142.085
## 35    635 135.82 138.45 137.135
## 36    637 126.17 128.87 127.520
## 37    640 130.85 133.02 131.935
```

5.3 Implied volatility for one option

Let's begin with the strike closest to the current stock price, roughly an at-the-money call.

```r
atm <- which.min(abs(SPY.option$Strike - SPY.S0))

K1 <- SPY.option$Strike[atm]
P1 <- SPY.option$Price[atm]

c(
  Spot = SPY.S0,
  Strike = K1,
  Midpoint = P1,
  Implied_volatility = implied.vol.call(SPY.S0, K1, T1, r, P1)
)
```

```
##               Spot             Strike           Midpoint Implied_volatility
##         769.480000         769.000000          13.155000           0.125935
```

5.4 Implied volatility across strikes

We can now repeat the same calculation for each option using `mapply()`. This lets us apply a function to two vectors at once: strikes and market prices.

```r
SPY.option$impliedVol <- mapply(
  function(K, P) implied.vol.call(SPY.S0, K, T1, r, P),
  K = SPY.option$Strike,
```

```
  P = SPY.option$Price
)

tail(SPY.option[c("Strike", "Price", "impliedVol")])
```

```
##     Strike Price impliedVol
## 213    880 0.015  0.1480901
## 214    885 0.015  0.1539492
## 215    890 0.015  0.1597169
## 216    895 0.015  0.1653930
## 217    900 0.015  0.1710690
## 218    905 0.015  0.1767451
```

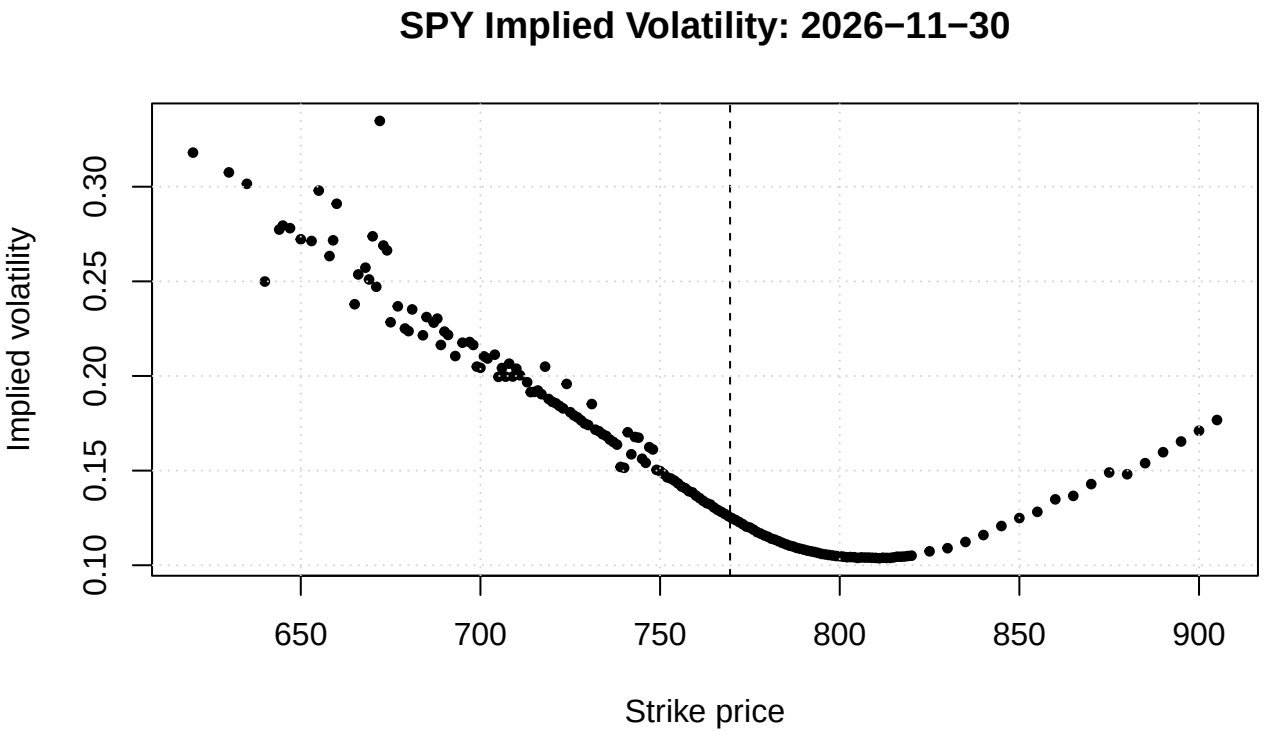
Some contracts may return `NA` when the market price cannot be matched within our volatility interval. This can happen with stale quotes or when the Black-Scholes assumptions are poor for a particular option.

5.5 Plot the volatility smile

```
smile <- subset(SPY.option, is.finite(impliedVol) & impliedVol > 0)

plot(
  smile$Strike, smile$impliedVol,
  pch = 19, cex = 0.6,
  xlab = "Strike price",
  ylab = "Implied volatility",
  main = paste("SPY Implied Volatility:", selected.expiration)
)

abline(v = SPY.S0, lty = 2)
grid()
```



The dashed line marks the current SPY price. If one constant volatility explained all market option prices, the points would lie along a horizontal line. Instead, implied volatility typically changes across strikes, producing a **volatility smile or skew**.

Not every irregular point necessarily represents a change in market expectations. Bid-ask spreads, stale quotes, and our simplified pricing model can all affect these estimates.

6 Summary

1. Black-Scholes turns volatility into an option price.
2. Implied volatility reverses that calculation through root-finding.
3. Bisection only requires prices; Newton-Raphson additionally uses Vega.
4. Applying the calculation across market strikes reveals the implied-volatility smile.